

An approach to storing data based on storing all values of a particular column together, rather than storing all attributes of a particular row together, has been found to work very well for analytical query processing. This idea, called *column-oriented storage*, is discussed in Section 13.6.

Some applications need very fast access to data and have small enough data sizes that the entire database can fit into the main memory of a database server machine. In such cases, we can keep a copy of the entire database in memory.¹ Databases that store the entire database in memory and optimize in-memory data structures as well as query processing and other algorithms used by the database to exploit the memory residency of data are called **main-memory databases**. Storage organization in main-memory databases is discussed in Section 13.7. We note that non-volatile memory that allows direct access to individual bytes or cache lines, called *storage class memory*, is under development. Main-memory database architectures can be further optimized for such storage.

13.2 File Organization

A database is mapped into a number of different files that are maintained by the underlying operating system. These files reside permanently on disks. A **file** is organized logically as a sequence of records. These records are mapped onto disk blocks. Files are provided as a basic construct in operating systems, so we shall assume the existence of an underlying *file system*. We need to consider ways of representing logical data models in terms of files.

Each file is also logically partitioned into fixed-length storage units called **blocks**, which are the units of both storage allocation and data transfer. Most databases use block sizes of 4 to 8 kilobytes by default, but many databases allow the block size to be specified when a database instance is created. Larger block sizes can be useful in some database applications.

A block may contain several records; the exact set of records that a block contains is determined by the form of physical data organization being used. We shall assume that *no record is larger than a block*. This assumption is realistic for most data-processing applications, such as our university example. There are certainly several kinds of large data items, such as images, that can be significantly larger than a block. We briefly discuss how to handle such large data items in Section 13.2.2, by storing large data items separately, and storing a pointer to the data item in the record.

In addition, we shall require that *each record is entirely contained in a single block*; that is, no record is contained partly in one block, and partly in another. This restriction simplifies and speeds up access to data items.

¹To be safe, not only should the current database fit in memory, but there should be a reasonable certainty that the database will continue to fit in memory in the medium term future, despite potential growth of the organization.

In a relational database, tuples of distinct relations are generally of different sizes. One approach to mapping the database to files is to use several files and to store records of only one fixed length in any given file. An alternative is to structure our files so that we can accommodate multiple lengths for records; however, files of fixed-length records are easier to implement than are files of variable-length records. Many of the techniques used for the former can be applied to the variable-length case. Thus, we begin by considering a file of fixed-length records and consider storage of variable-length records later.

13.2.1 Fixed-Length Records

As an example, let us consider a file of *instructor* records for our university database. Each record of this file is defined (in pseudocode) as:

```
type instructor = record
    ID varchar (5);
    name varchar(20);
    dept_name varchar (20);
    salary numeric (8,2);
end
```

Assume that each character occupies 1 byte and that numeric (8,2) occupies 8 bytes. Suppose that instead of allocating a variable amount of bytes for the attributes *ID*, *name*, and *dept_name*, we allocate the maximum number of bytes that each attribute can hold. Then, the *instructor* record is 53 bytes long. A simple approach is to use the first 53 bytes for the first record, the next 53 bytes for the second record, and so on (Figure 13.1).

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.1 File containing *instructor* records.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.2 File of Figure 13.1, with record 3 deleted and all records moved.

However, there are two problems with this simple approach:

1. Unless the block size happens to be a multiple of 53 (which is unlikely), some records will cross block boundaries. That is, part of the record will be stored in one block and part in another. It would thus require two block accesses to read or write such a record.
2. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored.

To avoid the first problem, we allocate only as many records to a block as would fit entirely in the block (this number can be computed easily by dividing the block size by the record size, and discarding the fractional part). Any remaining bytes of each block are left unused.

When a record is deleted, we could move the record that comes after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record has been moved ahead (Figure 13.2). Such an approach requires moving a large number of records. It might be easier simply to move the final record of the file into the space occupied by the deleted record (Figure 13.3).

It is undesirable to move records to occupy the space freed by a deleted record, since doing so requires additional block accesses. Since insertions tend to be more frequent than deletions, it is acceptable to leave open the space occupied by the deleted record and to wait for a subsequent insertion before reusing the space. A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.

At the beginning of the file, we allocate a certain number of bytes as a **file header**. The header will contain a variety of information about the file. For now, all we need

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Figure 13.3 File of Figure 13.1, with record 3 deleted and final record moved.

to store there is the address of the first record whose contents are deleted. We use this first record to store the address of the second available record, and so on. Intuitively, we can think of these stored addresses as *pointers*, since they point to the location of a record. The deleted records thus form a linked list, which is often referred to as a **free list**. Figure 13.4 shows the file of Figure 13.1, with the free list, after records 1, 4, and 6 have been deleted.

On insertion of a new record, we use the record pointed to by the header. We change the header pointer to point to the next available record. If no space is available, we add the new record to the end of the file.

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.4 File of Figure 13.1, with free list after deletion of records 1, 4, and 6.

Insertion and deletion for files of fixed-length records are simple to implement because the space made available by a deleted record is exactly the space needed to insert a record. If we allow records of variable-length in a file, this match no longer holds. An inserted record may not fit in the space left free by a deleted record, or it may fill only part of that space.

13.2.2 Variable-Length Records

Variable-length records arise in database systems due to several reasons. The most common reason is the presence of variable length fields, such as strings. Other reasons include record types that contain repeating fields such as arrays or multisets, and the presence of multiple record types within a file.

Different techniques for implementing variable-length records exist. Two different problems must be solved by any such technique:

- 1. How to represent a single record in such a way that individual attributes can be extracted easily, even if they are of variable length
- 2. How to store variable-length records within a block, such that records in a block can be extracted easily

The representation of a record with variable-length attributes typically has two parts: an initial part with fixed-length information, whose structure is the same for all records of the same relation, followed by the contents of variable-length attributes. Fixed-length attributes, such as numeric values, dates, or fixed-length character strings are allocated as many bytes as required to store their value. Variable-length attributes, such as varchar types, are represented in the initial part of the record by a pair (*offset*, *length*), where *offset* denotes where the data for that attribute begins within the record, and *length* is the length in bytes of the variable-sized attribute. The values for the variable-length attributes are stored consecutively, after the initial fixed-length part of the record. Thus, the initial part of the record stores a fixed size of information about each attribute, whether it is fixed-length or variable-length.

An example of such a record representation is shown in Figure 13.5. The figure shows an *instructor* record whose first three attributes *ID*, *name*, and *dept_name* are variable-length strings, and whose fourth attribute *salary* is a fixed-sized number. We assume that the offset and length values are stored in two bytes each, for a total of 4

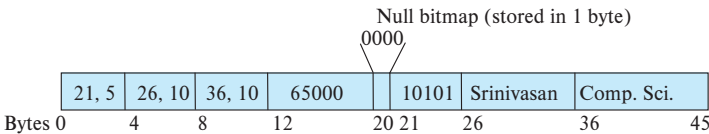


Figure 13.5 Representation of a variable-length record of the *instructor* relation.

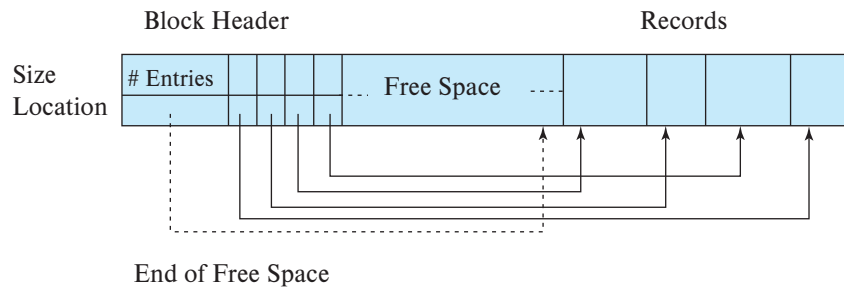


Figure 13.6 Slotted-page structure.

bytes per attribute. The *salary* attribute is assumed to be stored in 8 bytes, and each string takes as many bytes as it has characters.

The figure also illustrates the use of a **null bitmap**, which indicates which attributes of the record have a null value. In this particular record, if the *salary* were null, the fourth bit of the bitmap would be set to 1, and the *salary* value stored in bytes 12 through 19 would be ignored. Since the record has four attributes, the null bitmap for this record fits in 1 byte, although more bytes may be required with more attributes. In some representations, the null bitmap is stored at the beginning of the record, and for attributes that are null, no data (value, or offset/length) are stored at all. Such a representation would save some storage space, at the cost of extra work to extract attributes of the record. This representation is particularly useful for certain applications where records have a large number of fields, most of which are null.

We next address the problem of storing variable-length records in a block. The **slotted-page structure** is commonly used for organizing records within a block and is shown in Figure 13.6.² There is a header at the beginning of each block, containing the following information:

- The number of record entries in the header
- The end of free space in the block
- An array whose entries contain the location and size of each record

The actual records are allocated *contiguously* in the block, starting from the end of the block. The free space in the block is contiguous between the final entry in the header array and the first record. If a record is inserted, space is allocated for it at the end of free space, and an entry containing its size and location is added to the header.

If a record is deleted, the space that it occupies is freed, and its entry is set to *deleted* (its size is set to -1 , for example). Further, the records in the block before the deleted record are moved, so that the free space created by the deletion gets occupied, and all

²Here, “page” is synonymous with “block.”

free space is again between the final entry in the header array and the first record. The end-of-free-space pointer in the header is appropriately updated as well. Records can be grown or shrunk by similar techniques, as long as there is space in the block. The cost of moving the records is not too high, since the size of a block is limited: typical values are around 4 to 8 kilobytes.

The slotted-page structure requires that there be no pointers that point directly to records. Instead, pointers must point to the entry in the header that contains the actual location of the record. This level of indirection allows records to be moved to prevent fragmentation of space inside a block, while supporting indirect pointers to the record.

13.2.3 Storing Large Objects

Databases often store data that can be much larger than a disk block. For instance, an image or an audio recording may be multiple megabytes in size, while a video object may be multiple gigabytes in size. Recall that SQL supports the types **blob** and **clob**, which store binary and character large objects.

Many databases internally restrict the size of a record to be no larger than the size of a block.³ These databases allow records to logically contain large objects, but they store the large objects separate from the other (short) attributes of records in which they occur. A (logical) pointer to the object is then stored in the record containing the large object.

Large objects may be stored either as files in a file system area managed by the database, or as file structures stored in and managed by the database. In the latter case, such in-database large objects can optionally be represented using B⁺-tree file organizations, which we study in Section 14.4.1, to allow efficient access to any location within the object. B⁺-tree file organizations permit us to read an entire object, or specified byte ranges in the object, as well as to insert and delete parts of the object.

However, there are some performance issues with storing very large objects in databases. The efficiency of accessing large objects via database interfaces is one concern. A second concern is the size of database backups. Many enterprises periodically create “database dumps,” that is, backup copies of their databases; storing large objects in the database can result in a large increase in the size of the database dumps.

Many applications therefore choose to store very large objects, such as video data, outside of the database, in a file system. In such cases, the application may store the file name (usually a path in the file system) as an attribute of a record in the database. Storing data in files outside the database can result in file names in the database pointing to files that do not exist, perhaps because they have been deleted, which results in a form of foreign-key constraint violation. Further, database authorization controls are not applicable to data stored in the file system.

³This restriction helps simplify buffer management; as we see in Section 13.5, disk blocks are brought into an area of memory called the buffer before they are accessed. Records larger than a block would get split between blocks, which may be different areas of the buffer, and thus cannot be guaranteed to be in a contiguous area of memory.

Some databases support file system integration with the database, to ensure that constraints are satisfied (for example, deletion of files will be blocked if the database has a pointer to the file), and to ensure that access authorizations are enforced. Files can be accessed both from a file system interface and from the database SQL interface. For example, Oracle supports such integration through its SecureFiles and Database File System features.

13.3 Organization of Records in Files

So far, we have studied how records are represented in a file structure. A relation is a set of records. Given a set of records, the next question is how to organize them in a file. Several of the possible ways of organizing records in files are:

- **Heap file organization.** Any record can be placed anywhere in the file where there is space for the record. There is no ordering of records. Typically, there is either a single file or a set of files for each relation. Heap file organization is discussed in Section 13.3.1.
- **Sequential file organization.** Records are stored in sequential order, according to the value of a “search key” of each record. Section 13.3.2 describes this organization.
- **Multitable clustering file organization:** Generally, a separate file or set of files is used to store the records of each relation. However, in a *multitable clustering file organization*, records of several different relations are stored in the same file, and in fact in the same block within a file, to reduce the cost of certain join operations. Section 13.3.3 describes the multitable clustering file organization.
- **B⁺-tree file organization.** The traditional sequential file organization described in Section 13.3.2 does support ordered access even if there are insert, delete, and update operations, which may change the ordering of records. However, in the face of a large number of such operations, efficiency of ordered access suffers. We study another way of organizing records, called the *B⁺-tree file organization*, in Section 14.4.1. The B⁺-tree file organization is related to the B⁺-tree index structure described in that chapter and can provide efficient ordered access to records even if there are a large number of insert, delete, or update operations. Further, it supports very efficient access to specific records, based on the search key.
- **Hashing file organization.** A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed. Section 14.5 describes this organization; it is closely related to the indexing structures described in that chapter.

13.3.1 Heap File Organization

In a heap file organization, a record may be stored anywhere in the file corresponding to a relation. Once placed in a particular location, the record is not usually moved.⁴

When a record is inserted in a file, one option for choosing the location is to always add it at the end of the file. However, if records get deleted, it makes sense to use the space thus freed up to store new records. It is important for a database system to be able to efficiently find blocks that have free space, without having to sequentially search through all the blocks of the file.

Most databases use a space-efficient data structure called a **free-space map** to track which blocks have free space to store records. The free-space map is commonly represented by an array containing 1 entry for each block in the relation. Each entry represents a fraction f such that at least a fraction f of the space in the block is free. In PostgreSQL, for example, an entry is 1 byte, and the value stored in the entry must be divided by 256 to get the free-space fraction. The array is stored in a file, whose blocks are fetched into memory,⁵ as required. Whenever a record is inserted, deleted, or changed in size, if the occupancy fraction changes enough to affect the entry value, the entry is updated in the free-space map. An example of a free-space map for a file with 16 blocks is shown below. We assume that 3 bits are used to store the occupancy fraction; the value at position i should be divided by 8 to get the free-space fraction for block i .

4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

For example, a value of 7 indicates that at least $7/8$ th of the space in the block is free.

To find a block to store a new record of a given size, the database can scan the free-space map to find a block that has enough free space to store that record. If there is no such block, a new block is allocated for the relation.

While such a scan is much faster than actually fetching blocks to find free space, it can still be very slow for large files. To further speed up the task of locating a block with sufficient free space, we can create a second-level free-space map, which has, say 1 entry for every 100 entries of the main free-space map. That 1 entry stores the maximum value amongst the 100 entries in the main free-space map that it corresponds to. The free-space map below is a second level free-space map for our earlier example, with 1 entry for every 4 entries in the main free-space map.

4	7	2	6
---	---	---	---

⁴Records may be occasionally moved, for example, if the database sorts the records of the relation; but note that even if the relation is reordered by sorting, subsequent insertions and updates may result in the records no longer being ordered.

⁵Via the database buffer, which we discuss in Section 13.5.

With 1 entry for every 100 entries in the main free-space map, a scan of the second-level free space map would take only 1/100th of the time to scan the main free-space map; once a suitable entry indicating enough free space is found there, its corresponding 100 entries in the main free-space map can be examined to find a block with sufficient free space. Such a block must exist, since the second-level free-space map entry stores the maximum of the entries in the main free-space map. To deal with very large relations, we can create more levels beyond the second level, using the same idea.

Writing the free-space map to disk every time an entry in the map is updated would be very expensive. Instead, the free-space map is written periodically; as a result, the free-space map on disk may be outdated, and when a database starts up, it may get outdated data about available free space. The free-space map may, as a result, claim a block has free space when it does not; such an error will be detected when the block is fetched, and can be dealt with by a further search in the free-space map to find another block. On the other hand, the free-space map may claim that a block does not have free space when it does; generally this will not result in any problem other than unused free space. To fix any such errors, the relation is scanned periodically and the free-space map recomputed and written to disk.

13.3.2 Sequential File Organization

A **sequential file** is designed for efficient processing of records in sorted order based on some search key. A **search key** is any attribute or set of attributes; it need not be the primary key, or even a superkey. To permit fast retrieval of records in search-key order, we chain together records by pointers. The pointer in each record points to the next record in search-key order. Furthermore, to minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

Figure 13.7 shows a sequential file of *instructor* records taken from our university example. In that example, the records are stored in search-key order, using *ID* as the search key.

The sequential file organization allows records to be read in sorted order; that can be useful for display purposes, as well as for certain query-processing algorithms that we shall study in Chapter 15.

It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion. We can manage deletion by using pointer chains, as we saw previously. For insertion, we apply the following two rules:

1. Locate the record in the file that comes before the record to be inserted in search-key order.
2. If there is a free record (i.e., space left after a deletion) within the same block as this record, insert the new record there. Otherwise, insert the new record in

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

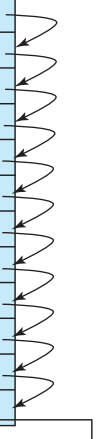


Figure 13.7 Sequential file for *instructor* records.

an *overflow block*. In either case, adjust the pointers so as to chain together the records in search-key order.

Figure 13.8 shows the file of Figure 13.7 after the insertion of the record (32222, Verdi, Music, 48000). The structure in Figure 13.8 allows fast insertion of new records, but it forces sequential file-processing applications to process records in an order that does not match the physical order of the records.

If relatively few records need to be stored in overflow blocks, this approach works well. Eventually, however, the correspondence between search-key order and physical order may be totally lost over a period of time, in which case sequential processing will become much less efficient. At this point, the file should be **reorganized** so that it is once again physically in sequential order. Such reorganizations are costly and must be done during times when the system load is low. The frequency with which reorganizations are needed depends on the frequency of insertion of new records. In the extreme case in which insertions rarely occur, it is possible always to keep the file in physically sorted order. In such a case, the pointer field in Figure 13.7 is not needed.

The B^+ -tree file organization, which we describe in Section 14.4.1, provides efficient ordered access even if there are many inserts, deletes, and updates, without requiring expensive reorganizations.

13.3.3 Multitable Clustering File Organization

Most relational database systems store each relation in a separate file, or a separate set of files. Thus, each file, and as a result, each block, stores records of only one relation, in such a design.

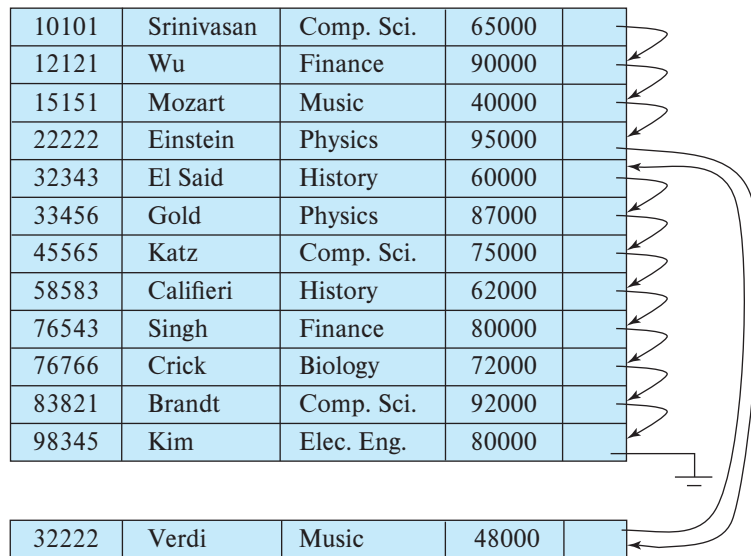


Figure 13.8 Sequential file after an insertion.

However, in some cases it can be useful to store records of more than one relation in a single block. To see the advantage of storing records of multiple relations in one block, consider the following SQL query for the university database:

```
select dept_name, building, budget, ID, name, salary
from department natural join instructor;
```

This query computes a join of the *department* and *instructor* relations. Thus, for each tuple of *department*, the system must locate the *instructor* tuples with the same value for *dept_name*. Ideally, these records will be located with the help of *indices*, which we shall discuss in Chapter 14. Regardless of how these records are located, however, they need to be transferred from disk into main memory. In the worst case, each record will reside on a different block, forcing us to do one block read for each record required by the query.

As a concrete example, consider the *department* and *instructor* relations of Figure 13.9 and Figure 13.10, respectively (for brevity, we include only a subset of the tuples

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

Figure 13.9 The *department* relation.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

Figure 13.10 The *instructor* relation.

of the relations we have used thus far). In Figure 13.11, we show a file structure designed for the efficient execution of queries involving the natural join of *department* and *instructor*. All the *instructor* tuples for a particular *dept_name* are stored near the *department* tuple for that *dept_name*. We say that the two relations are clustered on the key *dept_name*. We assume that each record contains the identifier of the relation to which it belongs, although this is not shown in Figure 13.11.

Although not depicted in the figure, it is possible to store the value of the *dept_name* attribute, which defines the clustering, only once for a group of tuples (from both relations), reducing storage overhead.

This structure allows for efficient processing of the join. When a tuple of the *department* relation is read, the entire block containing that tuple is copied from disk into main memory. Since the corresponding *instructor* tuples are stored on the disk near the *department* tuple, the block containing the *department* tuple contains tuples of the *instructor* relation needed to process the query. If a department has so many instructors that the *instructor* records do not fit in one block, the remaining records appear on nearby blocks.

A **multitable clustering file organization** is a file organization, such as that illustrated in Figure 13.11, that stores related records of two or more relations in each block.⁶ The **cluster key** is the attribute that defines which records are stored together; in our preceding example, the cluster key is *dept_name*.

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

Figure 13.11 Multitable clustering file structure.

⁶Note that the word *cluster* is often used to refer to a group of machines that together constitute a parallel database; that use of the word *cluster* is unrelated to the concept of multitable clustering.

Although a multitable clustering file organization can speed up certain join queries, it can result in slowing processing of other types of queries. For example, in our preceding example,

```
select *
from department;
```

requires more block accesses than it did in the scheme under which we stored each relation in a separate file, since each block now contains significantly fewer *department* records. To locate efficiently all tuples of the *department* relation within a particular block, we can chain together all the records of that relation using pointers; however, the number of blocks read does not get affected by using such chains.

When multitable clustering is to be used depends on the types of queries that the database designer believes to be most frequent. Careful use of multitable clustering can produce significant performance gains in query processing.

Multitable clustering is supported by the Oracle database system. Clusters can be created by using a **create cluster** command, with a specified cluster key. An extension of the **create table** command can be used to specify that a relation is to be stored in a specific cluster, with a particular attribute used as the cluster key. Multiple relations can thus be allocated to a cluster.

13.3.4 Partitioning

Many databases allow the records in a relation to be partitioned into smaller relations that are stored separately. Such **table partitioning** is typically done on the basis of an attribute value; for example, records in a *transaction* relation in an accounting database may be partitioned by year into smaller relations corresponding to each year, such as *transaction_2018*, *transaction_2019*, and so on. Queries can be written based on the *transaction* relation but are translated into queries on the year-wise relations. Most accesses are to records of the current year and include a selection based on the year. Query optimizers can rewrite such a query to only access the smaller relation corresponding to the requested year, and they can avoid reading records corresponding to other years. For example, a query

```
select *
from transaction
where year=2019
```

would only access the relation *transaction_2019*, ignoring the other relations, while a query without the selection condition would read all the relations.

The cost of some operations, such as finding free space for a record, increase with relation size; by reducing the size of each relation, partitioning helps reduce such overheads. Partitioning can also be used to store different parts of a relation on different

storage devices; for example, in the year 2019, *transaction_2018* and earlier year transactions can which are infrequently accessed could be stored on magnetic disk, while *transaction_2019* could be stored on SSD, for faster access.

13.4 Data-Dictionary Storage

So far, we have considered only the representation of the relations themselves. A relational database system needs to maintain data *about* the relations, such as the schema of the relations. In general, such “data about data” are referred to as **metadata**.

Relational schemas and other metadata about relations are stored in a structure called the **data dictionary** or **system catalog**. Among the types of information that the system must store are these:

- Names of the relations
- Names of the attributes of each relation
- Domains and lengths of attributes
- Names of views defined on the database, and definitions of those views
- Integrity constraints (e.g., key constraints)

In addition, many systems keep the following data on users of the system:

- Names of users, the default schemas of the users, and passwords or other information to authenticate users
- Information about authorizations for each user

Further, the database may store statistical and descriptive data about the relations and attributes, such as the number of tuples in each relation, or the number of distinct values for each attribute.

The data dictionary may also note the storage organization (heap, sequential, hash, etc.) of relations, and the location where each relation is stored:

- If relations are stored in operating system files, the dictionary would note the names of the file (or files) containing each relation.
- If the database stores all relations in a single file, the dictionary may note the blocks containing records of each relation in a data structure such as a linked list.

In Chapter 14, in which we study indices, we shall see a need to store information about each index on each of the relations:

- Name of the index

- Name of the relation being indexed
- Attributes on which the index is defined
- Type of index formed

All this metadata information constitutes, in effect, a miniature database. Some database systems store such metadata by using special-purpose data structures and code. It is generally preferable to store the data about the database as relations in the database itself. By using database relations to store system metadata, we simplify the overall structure of the system and harness the full power of the database for fast access to system data.

The exact choice of how to represent system metadata by relations must be made by the system designers. We show the schema diagram of a toy data dictionary in Figure 13.12, storing part of the information mentioned above. The schema is only illustrative; real implementations store far more information than what the figure shows. Read the manuals for whichever database you use to see what system metadata it maintains.

In the metadata representation shown, the attribute *index_attributes* of the relation *Index_metadata* is assumed to contain a list of one or more attributes, which can be represented by a character string such as “dept_name, building”. The *Index_metadata* relation is thus not in first normal form; it can be normalized, but the preceding representation is likely to be more efficient to access. The data dictionary is often stored in a nonnormalized form to achieve fast access.

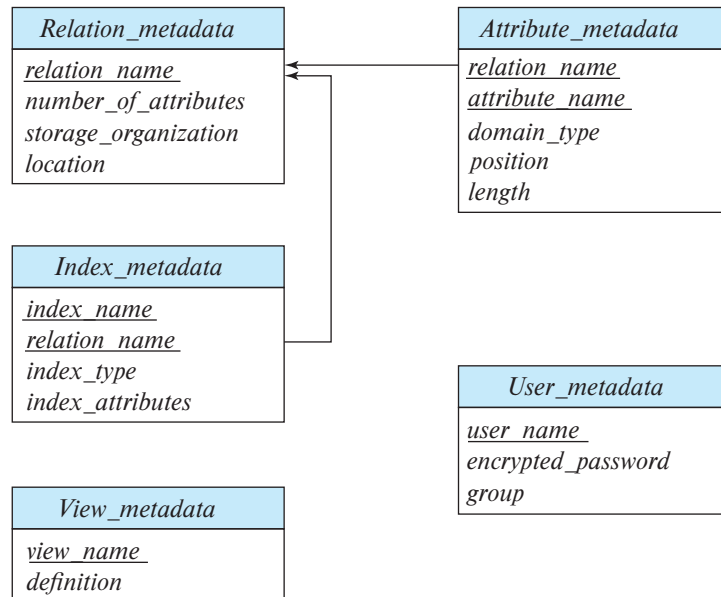


Figure 13.12 Relational schema representing part of the system metadata.

Whenever the database system needs to retrieve records from a relation, it must first consult the *Relation_metadata* relation to find the location and storage organization of the relation, and then fetch records using this information.

However, the storage organization and location of the *Relation_metadata* relation itself must be recorded elsewhere (e.g., in the database code itself, or in a fixed location in the database), since we need this information to find the contents of *Relation_metadata*.

Since system metadata are frequently accessed, most databases read it from the database into in-memory data structures that can be accessed very efficiently. This is done as part of the database startup, before the database starts processing any queries.

13.5 Database Buffer

The size of main memory on servers has increased greatly over the years, and many medium-sized databases can fit in memory. However, a server has many demands on its memory, and the amount of memory it can give to a database may be much smaller than the database size even for medium-sized databases. And many large databases are much larger than the available memory on servers.

Thus, even today, database data reside primarily on disk in most databases, and they must be brought into memory to be read or updated; updated data blocks must be written back to disk subsequently.

Since data access from disk is much slower than in-memory data access, a major goal of the database system is to minimize the number of block transfers between the disk and memory. One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory. The goal is to maximize the chance that, when a block is accessed, it is already in main memory, and, thus, no disk access is required.

Since it is not possible to keep all blocks in main memory, we need to manage the allocation of the space available in main memory for the storage of blocks. The **buffer** is that part of main memory available for storage of copies of disk blocks. There is always a copy kept on disk of every block, but the copy on disk may be a version of the block older than the version in the buffer. The subsystem responsible for the allocation of buffer space is called the **buffer manager**.

13.5.1 Buffer Manager

Programs in a database system make requests (i.e., calls) on the buffer manager when they need a block from disk. If the block is already in the buffer, the buffer manager passes the address of the block in main memory to the requester. If the block is not in the buffer, the buffer manager first allocates space in the buffer for the block, throwing out some other block, if necessary, to make space for the new block. The thrown-out block is written back to disk only if it has been modified since the most recent time that it was written to the disk. Then, the buffer manager reads in the requested block from the disk to the buffer, and passes the address of the block in main memory to the

requester. The internal actions of the buffer manager are transparent to the programs that issue disk-block requests.

If you are familiar with operating-system concepts, you will note that the buffer manager appears to be nothing more than a virtual-memory manager, like those found in most operating systems. One difference is that the size of the database might be larger than the hardware address space of a machine, so memory addresses are not sufficient to address all disk blocks. Further, to serve the database system well, the buffer manager must use techniques more sophisticated than typical virtual-memory management schemes:

13.5.1.1 Buffer replacement strategy

When there is no room left in the buffer, a block must be **evicted**, that is, removed, from the buffer before a new one can be read in. Most operating systems use a **least recently used (LRU)** scheme, in which the block that was referenced least recently is written back to disk and is removed from the buffer. This simple approach can be improved on for database applications(see Section 13.5.2).

13.5.1.2 Pinned blocks

Once a block has been brought into the buffer, a database process can read the contents of the block from the buffer memory. However, while the block is being read, if a concurrent process evicts the block and replaces it with a different block, the reader that was reading the contents of the old block will see incorrect data; if the block was being written when it was evicted, the writer would end up damaging the contents of the replacement block.

It is therefore important that before a process reads data from a buffer block, it ensures that the block will not get evicted. To do so, the process executes a **pin** operation on the block; the buffer manager never evicts a pinned block. When it has finished reading data, the process should execute an **unpin** operation, allowing the block to be evicted when required. The database code should be written carefully to avoid pinning too many blocks: if all the blocks in the buffer get pinned, no blocks can be evicted, and no other block can be brought into the buffer. If this happens, the database will be unable to carry out any further processing!

Multiple processes can read data from a block that is in the buffer. Each of them is required to execute a pin operation before accessing data, and an unpin after completing access. The block cannot be evicted until all processes that have executed a pin have then executed an unpin operation. A simple way to ensure this property is to keep a **pin count** for each buffer block. Each pin operation increments the count, and an unpin operation decrements the count. A page can be evicted only if the pin count equals 0.

13.5.1.3 Shared and Exclusive Locks on Buffer

A process that adds or deletes a tuple from a page may need to move the page contents around; during this period, no other process should read the contents of the page, since

they may be inconsistent. Database buffer managers allow processes to get shared and exclusive locks on the buffer.

We will study locking in more detail in Chapter 18, but here we discuss a limited form of locking in the context of the buffer manager. The locking system provided by the buffer manager allows a database process to lock a buffer block either in shared mode or in exclusive mode before accessing the block, and to release the lock later, after the access is completed. Here are the rules for locking:

- Any number of processes may have shared locks on a block at the same time.
- Only one process is allowed to get an exclusive lock at a time, and further when a process has an exclusive lock, no other process may have a shared lock on the block. Thus, an exclusive lock can be granted only when no other process has a lock on the buffer block.
- If a process requests an exclusive lock when a block is already locked in shared or exclusive mode, the request is kept pending until all earlier locks are released.
- If a process requests a shared lock when a block is not locked, or already shared locked, the lock may be granted; however, if another process has an exclusive lock, the shared lock is granted only after the exclusive lock has been released.

Locks are acquired and released as follows:

- Before carrying out any operation on a block, a process must pin the block as we saw earlier. Locks are obtained subsequently and must be released before unpinning the block.
- Before reading data from a buffer block, a process must get a shared lock on the block. When it is done reading the data, the process must release the lock.
- Before updating the contents of a buffer block, a process must get an exclusive lock on the block; the lock must be released after the update is complete.

These rules ensure that a block cannot be updated while another process is reading it, and conversely, a block cannot be read while another process is updating it. These rules are required for safety of buffer access; however, to protect a database system from problems due to concurrent access, these steps are not sufficient: further steps need to be taken. These are discussed further in Chapter 17 and Chapter 18.

13.5.1.4 Output of blocks

It is possible to output a block only when the buffer space is needed for another block. However, it makes sense to not wait until the buffer space is needed, but to rather write out updated blocks ahead of such a need. Then, when space is required in the buffer,

a block that has already been written out can be evicted, provided it is not currently pinned.

However, for the database system to be able to recover from crashes (Chapter 19), it is necessary to restrict those times when a block may be written back to disk. For instance, most recovery systems require that a block should not be written to disk while an update on the block is in progress. To enforce this requirement, a process that wishes to write the block to disk must obtain a shared lock on the block.

Most databases have a process that continually detects updated blocks and writes them back to disk.

13.5.1.5 Forced output of blocks

There are situations in which it is necessary to write a block to disk, to ensure that certain data on disk are in a consistent state. Such a write is called a **forced output** of a block. We shall see the reason for forced output in Chapter 19.

Memory contents and thus buffer contents are lost in a crash, whereas data on disk (usually) survive a crash. Forced output is used in conjunction with a logging mechanism to ensure that when a transaction that has performed updates commits, enough data has been written to disk to ensure the updates of the transaction are not lost. How exactly this is done is covered in detail in Chapter 19.

13.5.2 Buffer-Replacement Strategies

The goal of a replacement strategy for blocks in the buffer is to minimize accesses to the disk. For general-purpose programs, it is not possible to predict accurately which blocks will be referenced. Therefore, operating systems use the past pattern of block references as a predictor of future references. The assumption generally made is that blocks that have been referenced recently are likely to be referenced again. Therefore, if a block must be replaced, the least recently referenced block is replaced. This approach is called the **least recently used (LRU)** block-replacement scheme.

LRU is an acceptable replacement scheme in operating systems. However, a database system is able to predict the pattern of future references more accurately than an operating system. A user request to the database system involves several steps. The database system is often able to determine in advance which blocks will be needed by looking at each of the steps required to perform the user-requested operation. Thus, unlike operating systems, which must rely on the past to predict the future, database systems may have information regarding at least the short-term future.

To illustrate how information about future block access allows us to improve the LRU strategy, consider the processing of the SQL query:

```
select *
from instructor natural join department;
```

Assume that the strategy chosen to process this request is given by the pseudocode program shown in Figure 13.13. (We shall study other, more efficient, strategies in Chapter 15.)

Assume that the two relations of this example are stored in separate files. In this example, we can see that, once a tuple of *instructor* has been processed, that tuple is not needed again. Therefore, once processing of an entire block of *instructor* tuples is completed, that block is no longer needed in main memory, even though it has been used recently. The buffer manager should be instructed to free the space occupied by an *instructor* block as soon as the final tuple has been processed. This buffer-management strategy is called the **toss-immediate** strategy.

Now consider blocks containing *department* tuples. We need to examine every block of *department* tuples once for each tuple of the *instructor* relation. When processing of a *department* block is completed, we know that that block will not be accessed again until all other *department* blocks have been processed. Thus, the most recently used *department* block will be the final block to be re-referenced, and the least recently used *department* block is the block that will be referenced next. This assumption set is the exact opposite of the one that forms the basis for the LRU strategy. Indeed, the optimal strategy for block replacement for the above procedure is the **most recently used (MRU)** strategy. If a *department* block must be removed from the buffer, the MRU strategy chooses the most recently used block (blocks are not eligible for replacement while they are being used).

```

for each tuple i of instructor do
  for each tuple d of department do
    if i[dept_name] = d[dept_name]
      then begin
        let x be a tuple defined as follows:
        x[ID] := i[ID]
        x[dept_name] := i[dept_name]
        x[name] := i[name]
        x[salary] := i[salary]
        x[building] := d[building]
        x[budget] := d[budget]
        include tuple x as part of result of instructor ⋈ department
      end
    end
  end
end

```

Figure 13.13 Procedure for computing join.

For the MRU strategy to work correctly for our example, the system must pin the *department* block currently being processed. After the final *department* tuple has been processed, the block is unpinned, and it becomes the most recently used block.

In addition to using knowledge that the system may have about the request being processed, the buffer manager can use statistical information about the probability that a request will reference a particular relation. For example, the data dictionary, which we saw in Section 13.4, is one of the most frequently accessed parts of the database, since the processing of every query needs to access the data dictionary. Thus, the buffer manager should try not to remove data-dictionary blocks from main memory, unless other factors dictate that it do so. In Chapter 14, we discuss indices for files. Since an index for a file may be accessed more frequently than the file itself, the buffer manager should, in general, not remove index blocks from main memory if alternatives are available.

The ideal database block-replacement strategy needs knowledge of the database operations—both those being performed and those that will be performed in the future. No single strategy is known that handles all the possible scenarios well. Indeed, a surprisingly large number of database systems use LRU, despite that strategy's faults. The practice questions and exercises explore alternative strategies.

The strategy that the buffer manager uses for block replacement is influenced by factors other than the time at which the block will be referenced again. If the system is processing requests by several users concurrently, the concurrency-control subsystem (Chapter 18) may need to delay certain requests, to ensure preservation of database consistency. If the buffer manager is given information from the concurrency-control subsystem indicating which requests are being delayed, it can use this information to alter its block-replacement strategy. Specifically, blocks needed by active (nondelayed) requests can be retained in the buffer at the expense of blocks needed by the delayed requests.

The crash-recovery subsystem (Chapter 19) imposes stringent constraints on block replacement. If a block has been modified, the buffer manager is not allowed to write back the new version of the block in the buffer to disk, since that would destroy the old version. Instead, the block manager must seek permission from the crash-recovery subsystem before writing out a block. The crash-recovery subsystem may demand that certain other blocks be force-output before it grants permission to the buffer manager to output the block requested. In Chapter 19, we define precisely the interaction between the buffer manager and the crash-recovery subsystem.

13.5.3 Reordering of Writes and Recovery

Database buffers allow writes to be performed in-memory and output to disk at a later time, possibly in an order different from the order in which the writes were performed. File systems, too, routinely reorder write operations. However, such reordering can lead to inconsistent data on disk in the event of a system crash.

To understand the problem in the context of a file system, suppose that a file system uses a linked list to track which blocks are part of a file. Suppose also that it inserts a new node at the end of the list by first writing the data for the new node, then updating the pointer from the previous node. Suppose further that the writes were reordered, so the pointer was updated first, and the system crashes before the new node is written. The contents of the node would then be whatever happened to be on that disk earlier, resulting in a corrupted data structure.

To deal with the possibility of such data structure corruption, earlier-generation file systems had to perform a *file system consistency check* on system restart, to ensure that the data structures were consistent. And if they were not, extra steps had to be taken to restore them to consistency. These checks resulted in long delays in system restart after a crash, and the delays became worse as disk systems grew to higher capacities.

The file system can avoid inconsistencies in many cases if it writes updates to meta-data in a carefully chosen order. But doing so would mean that optimizations such as disk arm scheduling cannot be done, affecting the efficiency of the update. If a non-volatile write buffer were available, it could be used to perform the writes in order to non-volatile RAM and later reorder the writes when writing them to disk.

However, most disks do not come with a non-volatile write buffer; instead, modern file systems assign a disk for storing a log of the writes in the order that they are performed. Such a disk is called a **log disk**. For each write, the log contains the block number to be written to, and the data to be written, in the order in which the writes were performed. All access to the log disk is sequential, essentially eliminating seek time, and several consecutive blocks can be written at once, making writes to the log disk several times faster than random writes. As before, the data have to be written to their actual location on disk as well, but the write to the actual location can be done later; the writes can be reordered to minimize disk-arm movement.

If the system crashes before some writes to the actual disk location have been completed, when the system comes back up it reads the log disk to find those writes that had not been completed and carries them out then. After the writes have been performed, the records are deleted from the log disk.

File systems that support log disks as above are called **journaling file systems**. Journaling file systems can be implemented even without a separate log disk, keeping data and the log on the same disk. Doing so reduces the monetary cost at the expense of lower performance.

Most modern file systems implement journaling and use the log disk when writing file system metadata such as file allocation information. Journaling file systems allow quick restart without the need for such file system consistency checks.

However, writes performed by applications are usually not written to the log disk. Database systems instead implement their own forms of logging, which we study in Chapter 19, to ensure that the contents of a database can be safely recovered in the event of a failure, even if writes were reordered.